CASE STUDY DOCUMENT

Airline Ticket Booking System

SkyBooker

Search. Book. Fly. Effortlessly.

Version 1.0 | 2026

1. Synopsis

SkyBooker is a full-stack Airline Ticket Booking System inspired by MakeMyTrip and GoIbibo. It connects passengers with airlines, enabling them to search for one-way and round-trip flights, compare fare classes, select seats on an interactive seat map, enter passenger details, add ancillary services (meal preferences, extra baggage), and complete the booking with secure online payment — all from a unified platform.

The system supports four primary roles: Guests (who can search and browse flights without logging in), Passengers/Customers (who book, manage, and cancel tickets), Airline Staff (who manage flight schedules, seat inventory, and passenger manifests), and Platform Administrators (who oversee the entire ecosystem including airlines, airports, users, and analytics). Each role has a purpose-built dashboard with the controls appropriate to their function.

SkyBooker is architected as a microservices system modelled on the EShoppingZone reference pattern, with independent services for authentication/user management, flight search and management, seat map and inventory management, booking lifecycle management, passenger information management, payment processing, notification and alert management, and airline and airport master data management — all integrated through a Spring MVC website controller layer.

2. Detailed Requirements

2.1 General Requirements

- Guests can search and browse flights without logging in.
- Users must register or log in (email/password or Google OAuth) to book tickets.
- Role-based access control: Guest, Passenger, Airline Staff, and Admin.
- All users can update their profile and travel document details.
- In-app, email, and SMS notifications are dispatched for booking confirmation, check-in reminders, flight delays, and gate changes.
- Admin panel provides complete platform management including airline approvals, airport management, and analytics.

2.2 Passenger Requirements

- Register and log in via email/password or Google OAuth.
- Update profile including full name, phone number, passport number, and nationality.
- Search one-way flights by origin airport, destination airport, date, and number of passengers.
- Search round-trip flights specifying both departure and return dates.
- Filter search results by price range, airline, departure time, arrival time, number of stops, and seat class.

- View detailed flight information: airline, aircraft type, duration, stops, fare breakdown by class (Economy, Business, First).
- Select a fare class and view the interactive seat map for the chosen flight.
- Select available seats from the seat map (window, aisle, middle, extra legroom).
- Enter passenger details for each traveller: title, first/last name, date of birth, gender, passport number, nationality, and expiry date.
- Add meal preference (Veg/Non-Veg/Jain/Vegan) per passenger.
- Add extra baggage allowance (in kg) to the booking as a paid ancillary service.
- Review a complete booking summary with itemised fare breakdown before confirming.
- Make payment via credit/debit card, UPI, net banking, or wallet.
- Receive a booking confirmation email and SMS with the PNR code immediately after successful payment.
- Download the e-ticket and boarding pass as a PDF.
- View all upcoming and past bookings on the My Bookings dashboard.
- Retrieve a booking by PNR code without logging in.
- Cancel a booking and initiate a refund (subject to the airline's cancellation policy).
- Perform web check-in (24 hours before departure) and re-select seat if needed.
- Receive automated check-in reminder notifications 24 hours before departure.
- Receive flight delay, cancellation, and gate change alerts in real time.

2.3 Airline Staff Requirements

- Register as an airline operator and link to an approved airline entity on the platform.
- Add new flight schedules: flight number, origin, destination, departure/arrival times, aircraft type, and total seats.
- Edit existing flight schedules and update estimated times.
- Update flight status: On-Time, Delayed, Cancelled, Departed, or Arrived.
- Configure the seat map for a flight: add seats with class, row, column, features (window/aisle/extra legroom), and price multiplier.
- View the complete passenger manifest for any flight with passenger names, seat numbers, meal preferences, and check-in status.
- View flight-level revenue analytics: total bookings, revenue by fare class, seat utilisation.
- Send targeted flight delay or gate change alerts to all booked passengers on a specific flight.

2.4 Admin Requirements

- View and manage all user accounts: suspend, reactivate, or permanently delete.
- Create and manage airline profiles (IATA code, country, logo) and activate/deactivate airlines.
- Create and manage airport profiles (IATA/ICAO codes, city, country, timezone, GPS coordinates).
- View all bookings platform-wide with full lifecycle status and payment details.
- View all payment transactions and initiate refunds where applicable.

- Access platform analytics: total bookings, revenue, most popular routes, busiest airports, airline performance.
- Generate revenue reports for financial reconciliation and airline payouts.
- Send broadcast notifications to all users or by role (passengers, airline staff).
- View audit logs of all significant platform actions.

2.5 Booking Lifecycle Requirements

- Booking statuses: PENDING (payment not completed), CONFIRMED (payment successful), CANCELLED (by user or airline), COMPLETED (flight departed), NO_SHOW.
- A unique PNR (Passenger Name Record) code is generated at booking creation using a 6-character alphanumeric format.
- Seat reservation is held for 15 minutes during the booking process using optimistic locking; if payment is not completed, seats are auto-released.
- Fare calculation includes base fare + taxes (GST/fuel surcharge) + ancillary add-ons (baggage, meal), itemised in a FareSummary object.
- Web check-in opens 24 hours before departure and closes 1 hour before scheduled departure time.

2.6 Payment Requirements

- Passengers can pay via credit/debit card, UPI, net banking, or platform wallet.
- Payment processing is handled by an integrated payment gateway (Razorpay / Stripe).
- Each payment record stores amount, currency, mode, transaction ID, gateway response, and timestamps.
- On successful payment, the booking status transitions to CONFIRMED and an e-ticket is generated.
- Refunds are processed for eligible cancellations based on the airline's cancellation policy and credited to the original payment mode within 5–7 working days.
- Payment receipts are downloadable as PDF from the booking detail page.

2.7 Notification Requirements

- Notifications dispatched for: booking confirmed (email + SMS + app), check-in open reminder (24h before), flight delay/cancellation (push + email), gate change (push), boarding reminder (1h before).
- Each notification carries the relatedBookingId for deep-linking to the booking detail page.
- Multi-channel dispatch: in-app notification centre, email (JavaMailSender), and SMS (Twilio/AWS SNS).
- Airline staff can manually trigger flight delay or gate change alerts for their flights.
- Automated scheduler sends check-in reminders 24 hours before departure.

3. Use Case Diagram

The diagram below captures all actors and use cases within the SkyBooker platform — spanning flight search, booking lifecycle, seat selection, passenger management, payment, web check-in, notifications, airline schedule management, and administrative oversight.

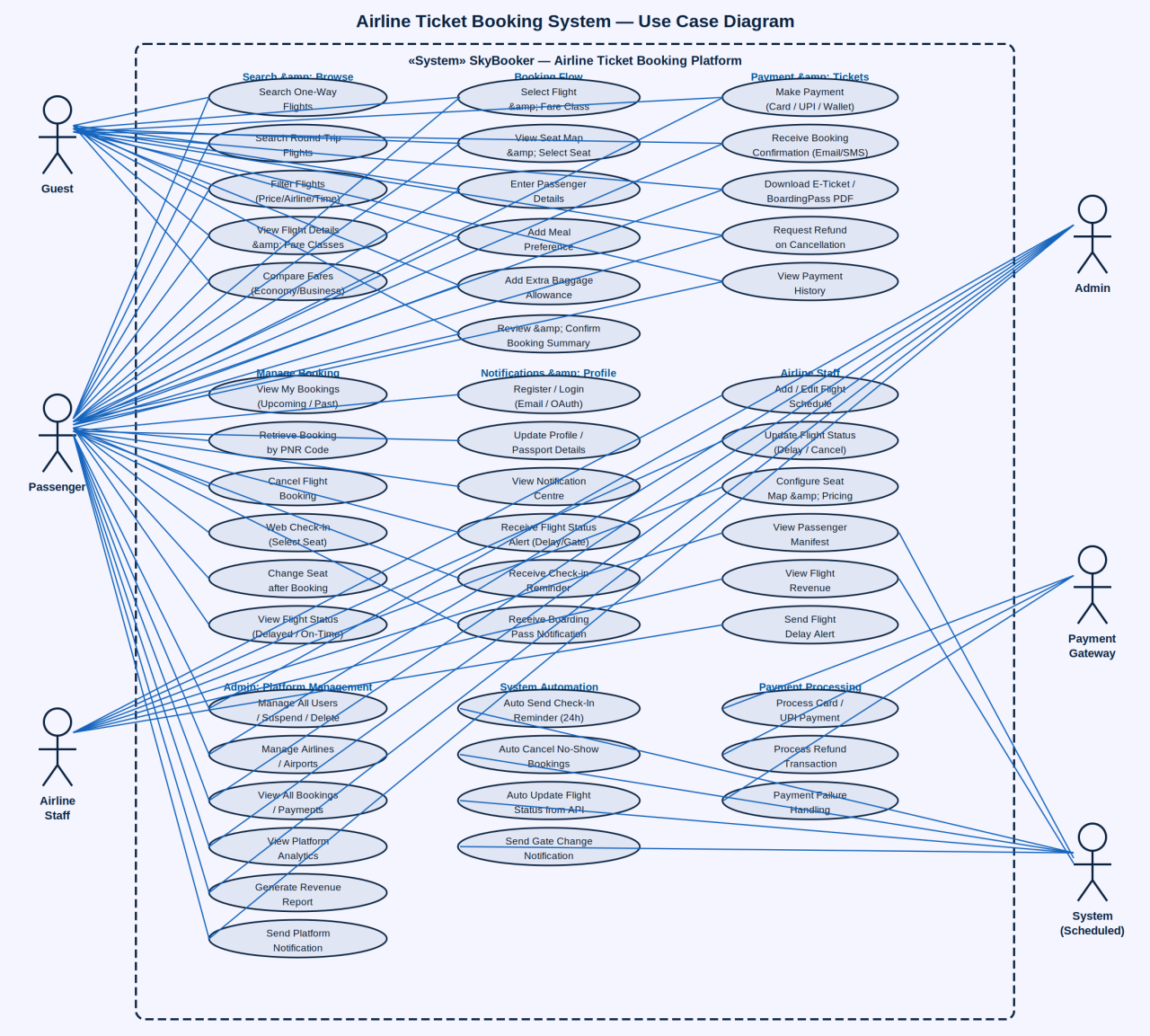

Figure 1: SkyBooker Airline Ticket Booking System — Complete Use Case Diagram

3.1 Actors

Actor	Description
Guest	Unauthenticated visitor who can search and browse flights without logging in
Passenger	Registered user who books, manages, and cancels flight tickets
Airline Staff	Airline operator who manages flight schedules, seat inventory, and passenger manifests
Admin	Platform administrator managing airlines, airports, users, and platform analytics
Payment Gateway	External service (Razorpay/Stripe) processing card, UPI, and net banking payments
System (Scheduled)	Automated component sending check-in reminders, auto-releasing unpaid seat holds, and updating flight status

3.2 Use Case Summary

Use Case	Actor(s)	Description
Search One-Way Flights	Guest, Passenger	Find available flights between two airports on a specified departure date for N passengers
Search Round-Trip Flights	Guest, Passenger	Find outbound and return flight pairs for a specified origin, destination, and two dates
Filter Flights	Guest, Passenger	Narrow results by price, airline, departure time, stops, and seat class
View Flight Details	Guest, Passenger	See fare breakdown by class, aircraft type, duration, and available seats per class
Compare Fares	Guest, Passenger	Side-by-side comparison of Economy, Business, and First Class prices and inclusions
Register / Login	Passenger	Create account or authenticate via email or Google OAuth
Update Profile	Passenger	Change name, phone, passport number, nationality, and travel document details
Select Flight & Fare Class	Passenger	Choose a flight and a specific fare class to proceed to seat selection
View Seat Map	Passenger	Open the interactive seat map and see available, occupied, and held seats by class
Select Seat	Passenger	Reserve a preferred seat (window, aisle, extra legroom) for each passenger
Enter Passenger Details	Passenger	Fill in title, name, date of birth, gender, passport, and nationality for each traveller

Use Case	Actor(s)	Description
Add Meal Preference	Passenger	Select a meal type (Veg/Non-Veg/Jain/Vegan) for each passenger
Add Extra Baggage	Passenger	Purchase additional baggage allowance in kg as a paid ancillary add-on
Review Booking Summary	Passenger	See itemised fare breakdown (base + taxes + ancillaries) before confirming
Make Payment	Passenger, Gateway	Pay via card, UPI, net banking, or wallet through the integrated payment gateway
Receive Booking Confirmation	Passenger	Get PNR code, e-ticket, and confirmation details via email and SMS immediately after payment
Download E-Ticket / PDF	Passenger	Download a formatted e-ticket and boarding pass PDF from the booking detail page
View My Bookings	Passenger	Browse all upcoming and past bookings with status and PNR details
Retrieve Booking by PNR	Guest, Passenger	Look up any booking using its 6-character PNR code without logging in
Cancel Booking	Passenger	Cancel a confirmed booking; trigger refund based on the airline's cancellation policy
Request Refund	Passenger, Gateway	Initiate a refund transaction for a cancelled booking to the original payment mode
View Payment History	Passenger	See all payment transactions with amounts, modes, and status
Web Check-In	Passenger	Complete online check-in within the 24h–1h pre-departure window
Change Seat after Booking	Passenger	Re-select a seat from the seat map after booking and before check-in closes
View Flight Status	Guest, Passenger	See real-time status (On-Time, Delayed, Cancelled, Departed) for any flight
View Notification Centre	Passenger	Browse all in-app alerts with read/unread status and booking deep-links
Receive Flight Status Alert	Passenger, System	In-app and push notification for flight delays, cancellations, and gate changes
Receive Check-In Reminder	Passenger, System	Automated alert 24 hours before scheduled departure opening web check-in
Receive Boarding Pass Alert	Passenger	Notification with boarding pass ready to download after successful check-in
Add / Edit Flight Schedule	Airline Staff	Create or update a flight with number, route, times, aircraft type, and total seats

Use Case	Actor(s)	Description
Update Flight Status	Airline Staff	Mark a flight as Delayed, Cancelled, Departed, or Arrived in real time
Configure Seat Map	Airline Staff	Define seat layout with classes, rows, columns, features, and price multipliers
View Passenger Manifest	Airline Staff	Access the full passenger list for a flight with seat, meal, and check-in status
View Flight Revenue	Airline Staff	See total bookings, revenue by fare class, and seat utilisation for any flight
Send Flight Delay Alert	Airline Staff	Manually trigger a delay or gate change notification to all booked passengers
Manage All Users	Admin	View, suspend, reactivate, or permanently delete any user account
Manage Airlines	Admin	Create, edit, activate, or deactivate airline profiles with IATA code and logo
Manage Airports	Admin	Create and update airport profiles with IATA/ICAO codes, city, country, and timezone
View All Bookings	Admin	Monitor every booking platform-wide with full lifecycle status
View Platform Analytics	Admin	Access total bookings, revenue, popular routes, and airline performance metrics
Generate Revenue Report	Admin	Export financial data for airline payouts and platform commission reconciliation
Send Platform Notification	Admin	Broadcast an alert to all users or by role (passengers/airline staff)
Auto-Release Seat Hold	System	Release a seat reservation if payment is not completed within 15 minutes
Auto Cancel No-Show	System	Transition un-checked-in bookings to NO_SHOW status after gate closure
Auto Update Flight Status	System	Sync flight departure/arrival status from airline API at scheduled intervals

4. Class Diagrams — All Services

SkyBooker follows the same layered microservices architecture as the EShoppingZone reference system. Each service comprises five layers: Entity/POJO (domain model), Repository Interface (data access), Service Interface (business contract), ServiceImpl (business logic), and REST Resource (API exposure). The ten class diagrams below detail every service in the platform.

4.1 Auth / User-Service

The Auth/User-Service manages all user identity operations on the platform. Users carry passportNumber and nationality fields in addition to standard profile fields, enabling airline compliance checks. The role field (PASSENGER/AIRLINE_STAFF/ADMIN) gates access to the booking dashboard, airline operations panel, and admin panel respectively. JWT tokens are issued on login and validated on every protected API call.

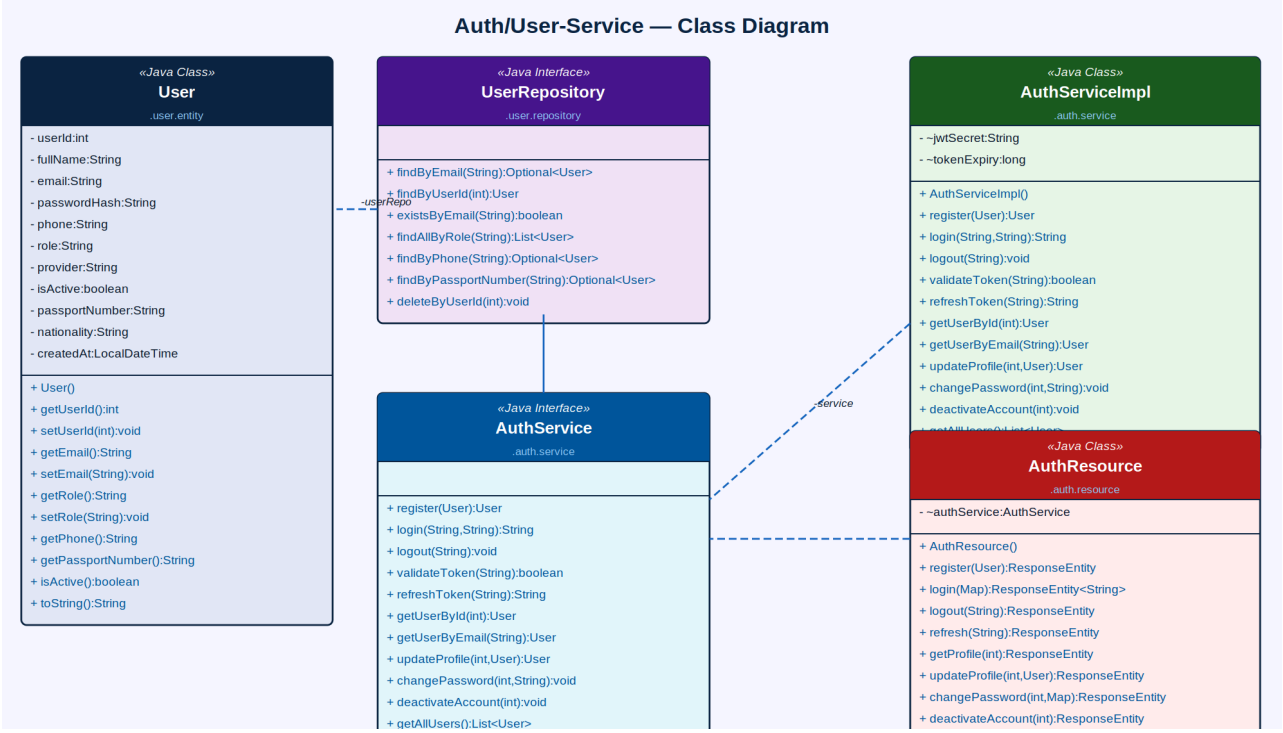

Figure 2: Auth/User-Service — Class Diagram

Component	Type	Responsibility
User	Java Class (Entity)	Stores userId, fullName, email, passwordHash, phone, role (PASSENGER/STAFF/ADMIN), provider, isActive, passportNumber, nationality, createdAt
UserRepository	Java Interface	findByEmail(), findByUserId(), existsByEmail(), findAllByRole(), findByPhone(), findByPassportNumber(), deleteByUserId()
AuthServiceImpl	Java Class	Implements register, login, logout, JWT generation/validation, OAuth2 handling, profile update, password change, account deactivation, getAllUsers()
AuthService	Java Interface	Declares register(), login(), logout(), validateToken(), refreshToken(), getUserById(),

Component	Type	Responsibility
		updateProfile(), changePassword(), deactivateAccount(), getAllUsers()
AuthResource	Java Class (REST)	Exposes /auth/register, /auth/login, /auth/logout, /auth/refresh, /auth/profile (GET/PUT), /auth/password, /auth/deactivate, /auth/users

4.2 Flight-Service

The Flight-Service manages the flight schedule inventory. Each Flight stores the complete schedule including departure/arrival times, duration, aircraft type, and the current availableSeats counter which is decremented atomically on booking creation and incremented on cancellation. The searchFlights operation supports both one-way queries and is called twice (for outbound and return) in a round-trip search. The updateStatus operation triggers the Notification-Service to alert all booked passengers on status changes.

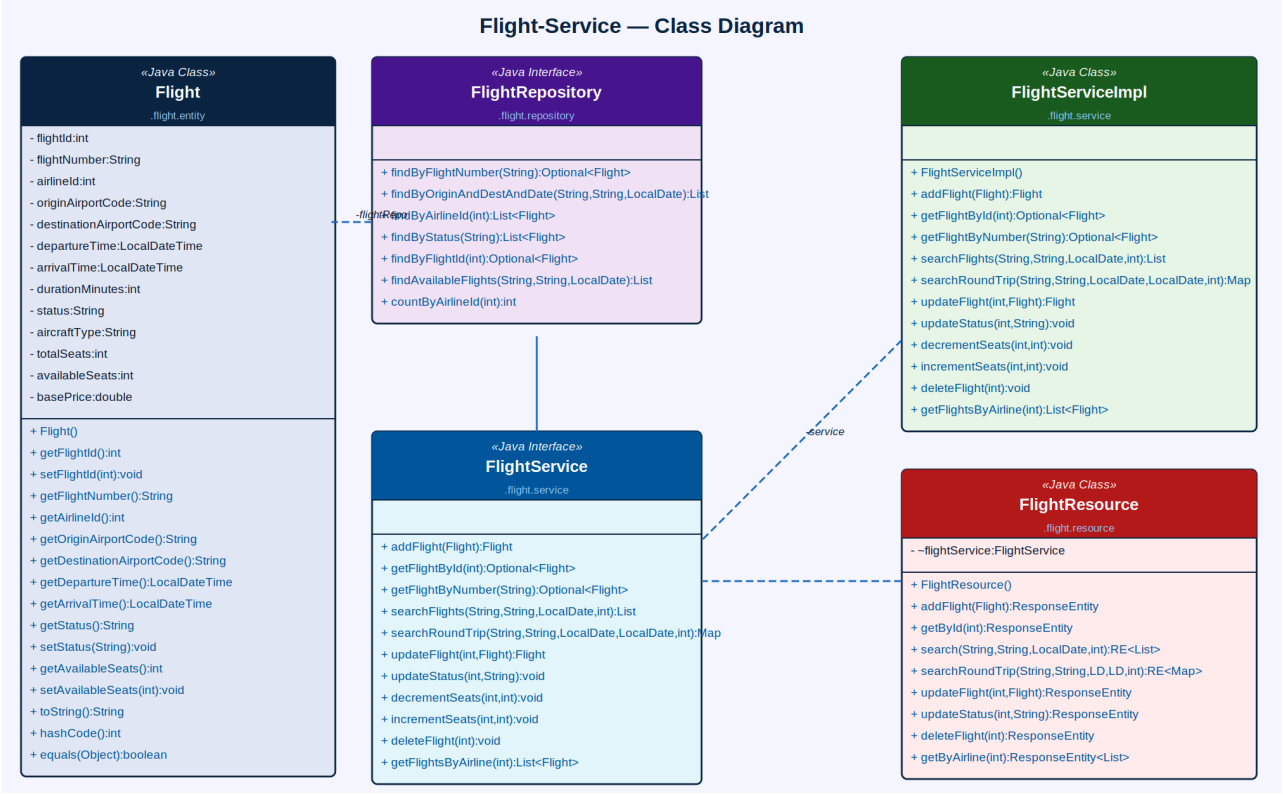

Figure 3: Flight-Service — Class Diagram

Component	Type	Key Attributes / Methods
Flight	Java Class (Entity)	flightId, flightNumber, airlineId, originAirportCode, destinationAirportCode, departureTime, arrivalTime, durationMinutes, status (ON_TIME/DELAYED/CANCELLED/DEPARTED/ARRIVED), aircraftType, totalSeats, availableSeats, basePrice
FlightRepository	Java Interface	findByFlightNumber(), findByOriginAndDestAndDate(), findByAirlineId(), findByStatus(), findByFlightId(), findAvailableFlights(), countByAirlineId()

Component	Type	Key Attributes / Methods
FlightServiceImpl	Java Class	addFlight(), getFlightById(), getFlightByNumber(), searchFlights(), searchRoundTrip(), updateFlight(), updateStatus(), decrementSeats(), incrementSeats(), deleteFlight(), getFlightsByAirline()
FlightService	Java Interface	Declares all flight CRUD, one-way/round-trip search, status update, and atomic seat counter management operations
FlightResource	Java Class (REST)	Exposes /flights endpoints: POST (add), GET (by id/number/airline), GET search (one-way/round-trip), PUT (update/status), DELETE

4.3 Seat / Aircraft-Service

The Seat/Aircraft-Service manages the seat inventory for each flight. Each Seat carries class (ECONOMY/BUSINESS/FIRST), position metadata (row, column, window/aisle flags), an extra legroom flag, and a priceMultiplier applied on top of the base fare. Seat status transitions: AVAILABLE → HELD (during 15-minute payment window) → CONFIRMED (payment successful) or back to AVAILABLE (payment timeout). Optimistic locking ensures no two sessions can hold the same seat simultaneously.

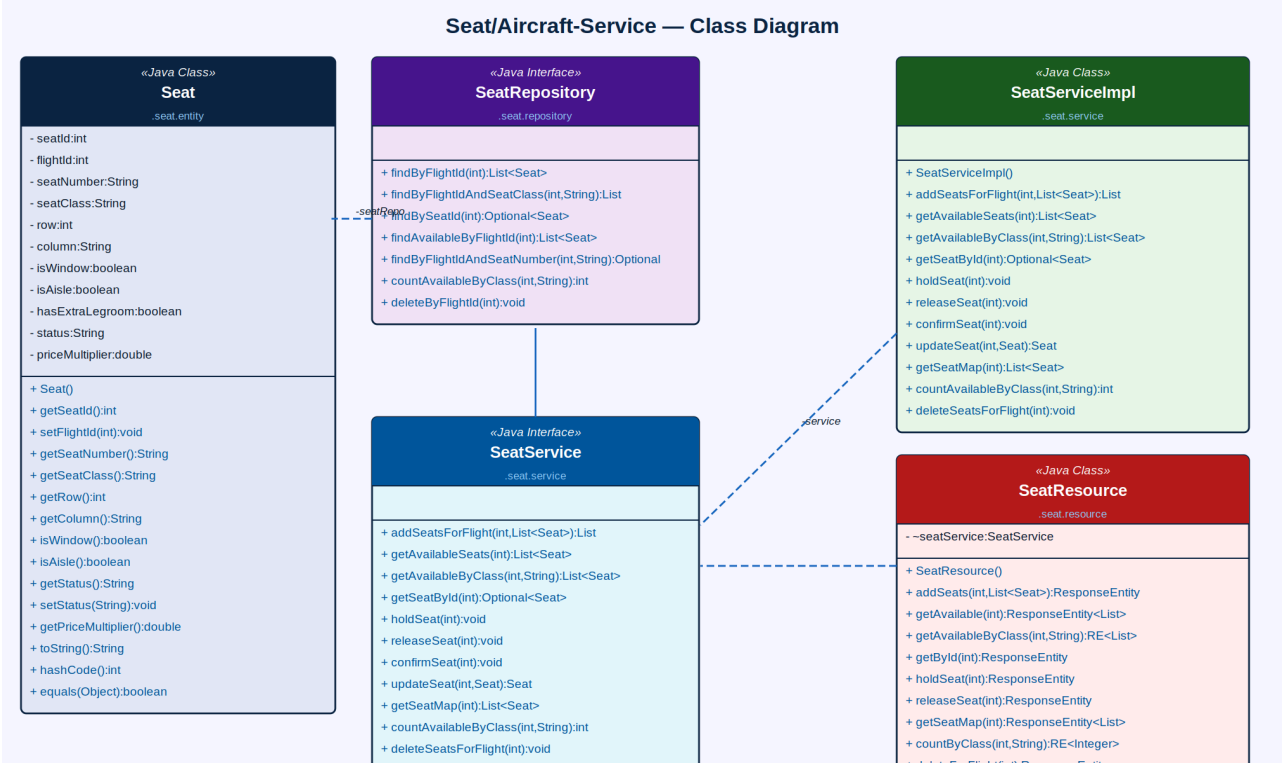

Figure 4: Seat/Aircraft-Service — Class Diagram

Component	Type	Key Attributes / Methods
Seat	Java Class (Entity)	seatId, flightId, seatNumber (e.g. 12A), seatClass (ECONOMY/BUSINESS/FIRST), row, column, isWindow, isAisle, hasExtraLegroom, status (AVAILABLE/HELD/CONFIRMED/BLOCKED), priceMultiplier
SeatRepository	Java Interface	findByFlightId(), findByFlightIdAndSeatClass(), findBySeatId(), findAvailableByFlightId(), findByFlightIdAndSeatNumber(), countAvailableByClass(), deleteByFlightId()
SeatServiceImpl	Java Class	addSeatsForFlight(), getAvailableSeats(), getAvailableByClass(), getSeatById(), holdSeat(), releaseSeat(), confirmSeat(), updateSeat(),

Component	Type	Key Attributes / Methods
		getSeatMap(), countAvailableByClass(), deleteSeatsForFlight()
SeatService	Java Interface	Declares seat creation, availability queries, hold/release/confirm lifecycle, seat map retrieval, and count operations
SeatResource	Java Class (REST)	Exposes /seats endpoints: POST (addSeats), GET (available/by class/map/id), PUT (hold/release/update), GET count, DELETE (for flight)

4.4 Booking-Service

The Booking-Service is the central orchestration service of the platform. It creates bookings with a system-generated PNR code, coordinates seat holds with the Seat-Service, triggers payment initiation via the Payment-Service, and dispatches confirmation notifications via the Notification-Service. The calculateFare operation returns a FareSummary DTO with base fare, taxes, and ancillary costs. Booking cancellation releases held seats, updates available seat counters, and triggers refund processing.

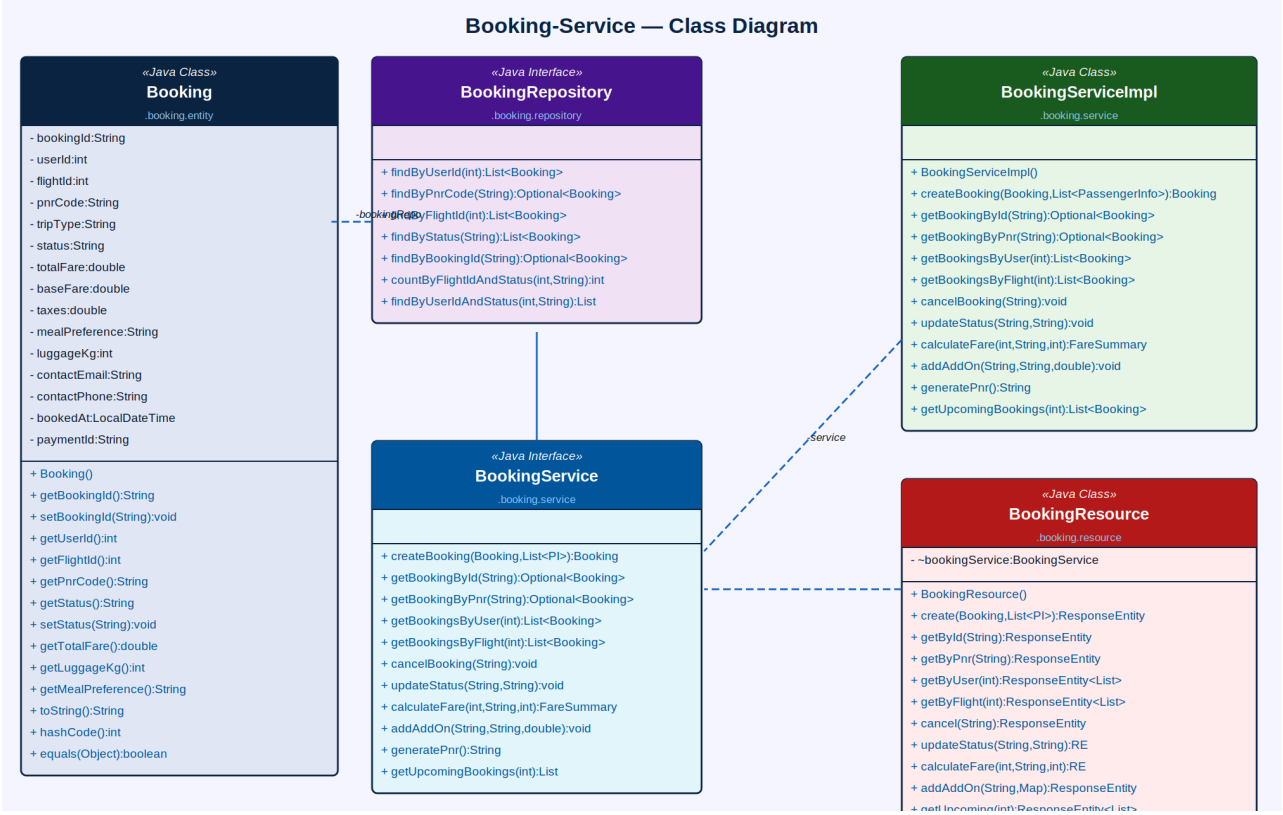

Figure 5: Booking-Service — Class Diagram

Component	Type	Key Attributes / Methods
Booking	Java Class (Entity)	bookingId (UUID), userId, flightId, pnrCode (6-char alphanumeric), tripType (ONE_WAY/ROUND_TRIP), status (PENDING/CONFIRMED/CANCELLED/COMPLETED/NO_SHOW), totalFare, baseFare, taxes, mealPreference, luggageKg, contactEmail, contactPhone, bookedAt, paymentId
BookingRepository	Java Interface	findByUserId(), findByPnrCode(), findByFlightId(), findByStatus(), findByBookingId(), countByFlightIdAndStatus(), findByUserIdAndStatus()

Component	Type	Key Attributes / Methods
BookingServiceImpl	Java Class	createBooking(), getBookingById(), getBookingByPnr(), getBookingsByUser(), getBookingsByFlight(), cancelBooking(), updateStatus(), calculateFare(), addAddOn(), generatePnr(), getUpcomingBookings()
BookingService	Java Interface	Declares booking creation with passenger list, PNR retrieval, cancellation, fare calculation, add-on management, and upcoming-bookings query
BookingResource	Java Class (REST)	Exposes /bookings endpoints: POST (create), GET (by id/pnr/user/flight/upcoming), PUT (cancel/status), GET calculateFare, POST addAddOn

4.5 Passenger-Service

The Passenger-Service stores the detailed travel information for each individual traveller on a booking. Each PassengerInfo is linked to a booking and carries a unique ticket number generated at booking time. The service validates passenger data (age eligibility, passport expiry) before booking confirmation. After web check-in, the assigned seatId and seatNumber fields are updated. PassengerType (ADULT/CHILD/INFANT) determines age-based fare calculations.

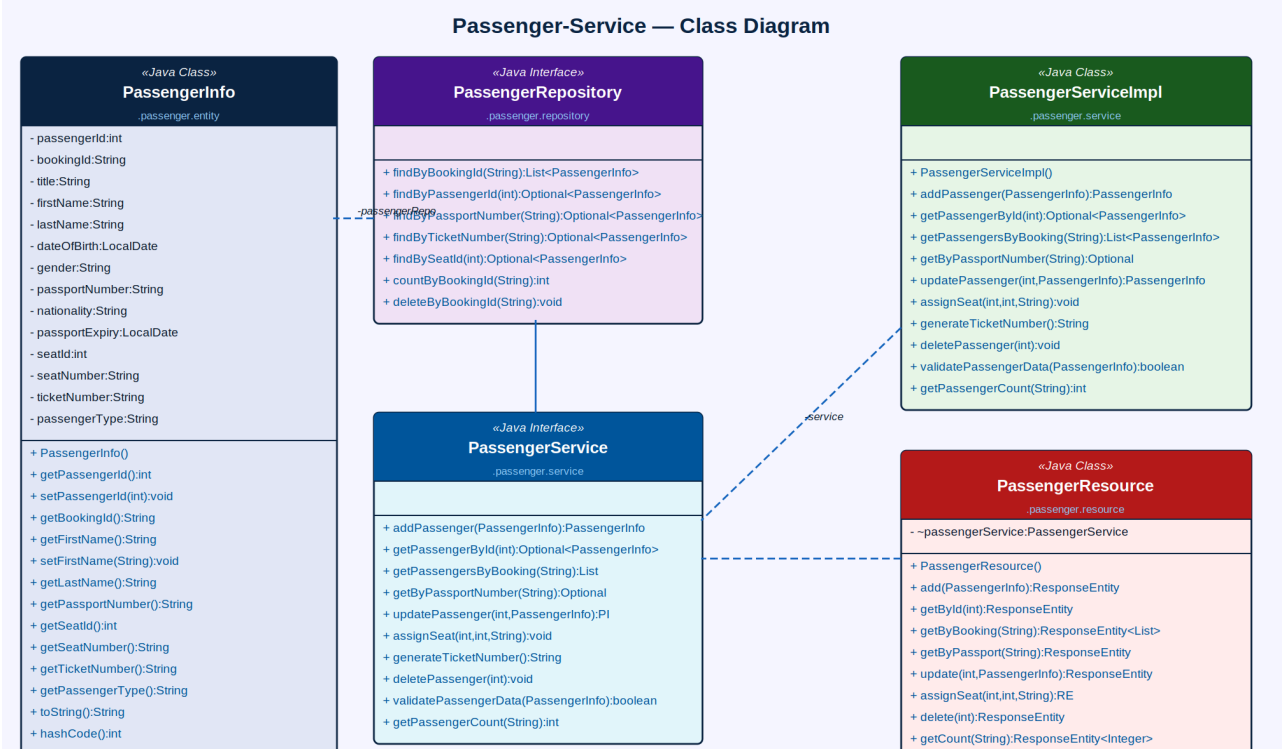

Figure 6: Passenger-Service — Class Diagram

Component	Type	Key Attributes / Methods
PassengerInfo	Java Class (Entity)	passengerId, bookingId, title, firstName, lastName, dateOfBirth, gender, passportNumber, nationality, passportExpiry, seatId, seatNumber, ticketNumber (generated), passengerType (ADULT/CHILD/INFANT)
PassengerRepository	Java Interface	findByBookingId(), findByPassengerId(), findByPassportNumber(), findByTicketNumber(), findBySeatId(), countByBookingId(), deleteByBookingId()
PassengerServiceImpl	Java Class	addPassenger(), getPassengerById(), getPassengersByBooking(), getByPassportNumber(), updatePassenger(), assignSeat(), generateTicketNumber(),

Component	Type	Key Attributes / Methods
		deletePassenger(), validatePassengerData(), getPassengerCount()
PassengerService	Java Interface	Declares all passenger CRUD, seat assignment, ticket number generation, validation, and count operations
PassengerResource	Java Class (REST)	Exposes /passengers endpoints: POST (add), GET (by id/booking/passport), PUT (update/assignSeat), DELETE, GET count

4.6 Payment-Service

The Payment-Service processes all financial transactions for the platform. The initiatePayment operation creates a pending payment record and returns a payment session for the frontend to redirect to the gateway. On webhook callback from Razorpay/Stripe, processPayment updates the status and triggers the Booking-Service to confirm the booking. Refunds are processed through the same gateway and credited to the original payment mode. The generateReceipt operation produces a downloadable payment receipt.

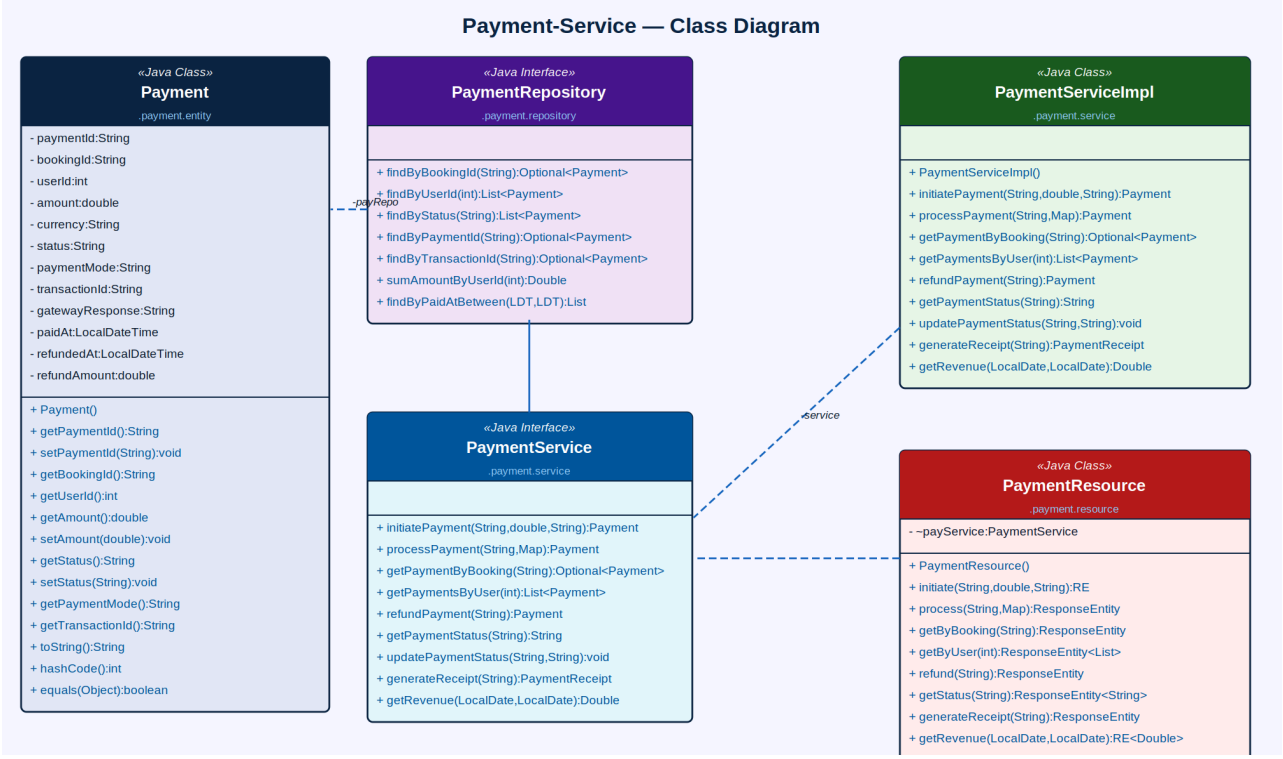

Figure 7: Payment-Service — Class Diagram

Component	Type	Key Attributes / Methods
Payment	Java Class (Entity)	paymentId (UUID), bookingId, userId, amount, currency, status (PENDING/PAID/FAILED/REFUNDED), paymentMode (CARD/UPI/NETBANKING/WALLET), transactionId, gatewayResponse, paidAt, refundedAt, refundAmount
PaymentRepository	Java Interface	findByBookingId(), findByUserId(), findByStatus(), findByPaymentId(), findByTransactionId(), sumAmountByUserId(), findByPaidAtBetween()
PaymentServiceImpl	Java Class	initiatePayment(), processPayment(), getPaymentByBooking(), getPaymentsByUser(),

Component	Type	Key Attributes / Methods
		refundPayment(), getPaymentStatus(), updatePaymentStatus(), generateReceipt(), getRevenue()
PaymentService	Java Interface	Declares payment initiation, gateway callback processing, refund, status query, receipt generation, and revenue aggregation operations
PaymentResource	Java Class (REST)	Exposes /payments endpoints: POST (initiate/process/refund), GET (by booking/user/status), GET (generateReceipt), GET revenue

4.7 Notification-Service

The Notification-Service dispatches multi-channel alerts for all key booking and flight lifecycle events. The sendBookingConfirmation operation triggers an email with the e-ticket PDF attached, an SMS with the PNR code, and an in-app notification with a booking deep-link. Flight status alerts are dispatched to all passengers on a specific flight. A scheduled job runs hourly to identify bookings with departures 24 hours away and send check-in reminder notifications.

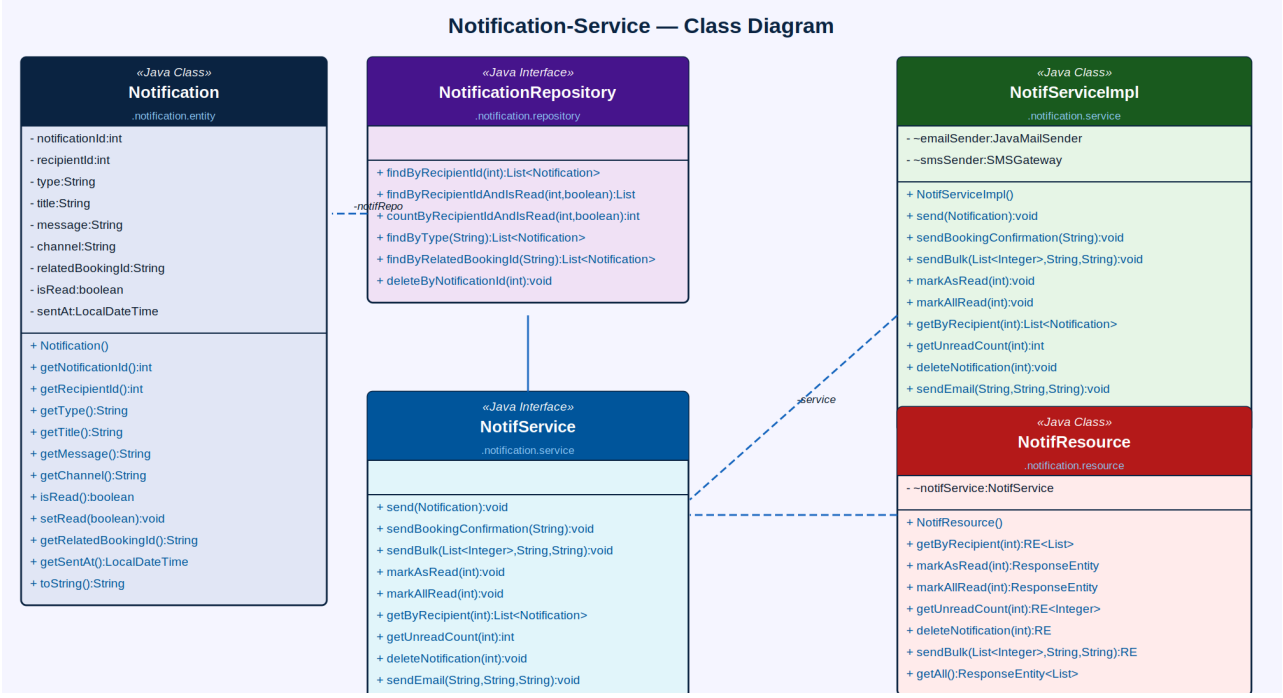

Figure 8: Notification-Service — Class Diagram

Component	Type	Key Attributes / Methods
Notification	Java Class (Entity)	notificationId, recipientId, type (BOOKING_CONFIRMED/FLIGHT_DELAY/GATE_CHANGE/CHECKIN_REMINDER/BOARDING/CANCELLATION), title, message, channel (APP/EMAIL/SMS), relatedBookingId, isRead, sentAt
NotificationRepository	Java Interface	findByRecipientId(), findByRecipientIdAndIsRead(), countByRecipientIdAndIsRead(), findByType(), findByRelatedBookingId(), deleteByNotificationId()
NotifServiceImpl	Java Class	send(), sendBookingConfirmation(), sendBulk(), markAsRead(), markAllRead(), getByRecipient(), getUnreadCount(), deleteNotification(), sendEmail(), sendSMS()

Component	Type	Key Attributes / Methods
NotifService	Java Interface	Declares single/bulk notification dispatch, booking confirmation trigger, email/SMS channels, read-state management, and deletion operations
NotifResource	Java Class (REST)	Exposes /notifications endpoints: getByRecipient, markAsRead, markAllRead, getUnreadCount, delete, sendBulk, getAll

4.8 Airline & Airport-Service

The Airline & Airport-Service manages the master data for airlines and airports. Airlines are identified by their IATA code (e.g., 6E for IndiGo, AI for Air India). Airports store IATA/ICAO codes, city, country, GPS coordinates, and timezone for accurate local-time display. The service is primarily managed by Admin; airline staff access it read-only for flight management. The searchAirports operation powers the autocomplete search on the flight search form.

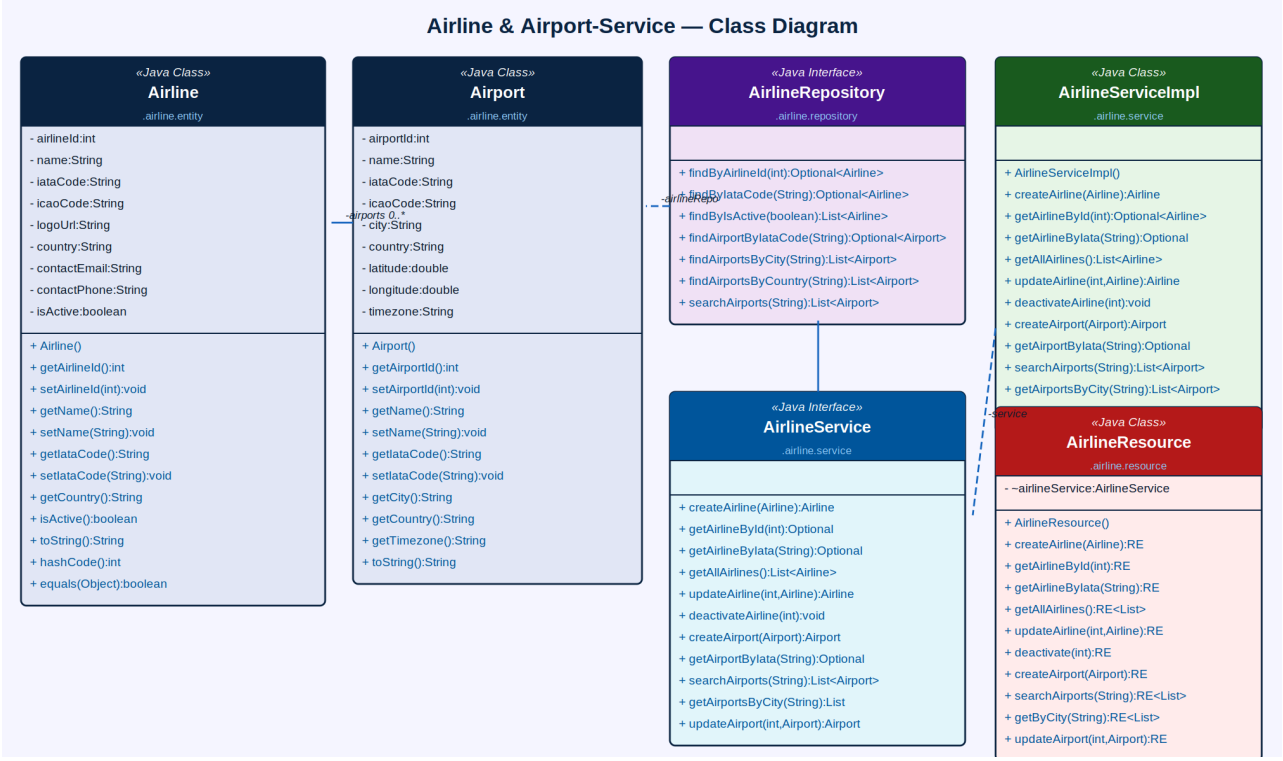

Figure 9: Airline & Airport-Service — Class Diagram

Component	Type	Key Attributes / Methods
Airline	Java Class (Entity)	airlineId, name, iataCode (unique), icaoCode, logoUrl, country, contactEmail, contactPhone, isActive
Airport	Java Class (Entity)	airportId, name, iataCode (unique), icaoCode, city, country, latitude, longitude, timezone — linked to Airlines (0..*)
AirlineRepository	Java Interface	findByAirlineId(), findByIataCode(), findByIsActive(), findAirportByIataCode(), findAirportsByCity(), findAirportsByCountry(), searchAirports()
AirlineServiceImpl	Java Class	createAirline(), getAirlineById(), getAirlineByIata(), getAllAirlines(), updateAirline(), deactivateAirline(),

Component	Type	Key Attributes / Methods
		createAirport(), getAirportByIata(), searchAirports(), getAirportsByCity(), updateAirport()
AirlineService	Java Interface	Declares all airline and airport CRUD, IATA-based lookup, search, and deactivation operations
AirlineResource	Java Class (REST)	Exposes /airlines and /airports endpoints: POST, GET (by id/iata/city/search/all), PUT (update/deactivate)

4.9 Website-Controller (MVC Layer)

The Website-Controller is the Spring MVC integration layer that wires all underlying microservices and renders Thymeleaf views for passengers, airline staff, and administrators. It contains three controllers — CustomerController (full passenger booking journey), AirlineController (airline operations dashboard), and AdminController (admin panel) — each injecting relevant service dependencies via Spring @Autowired.

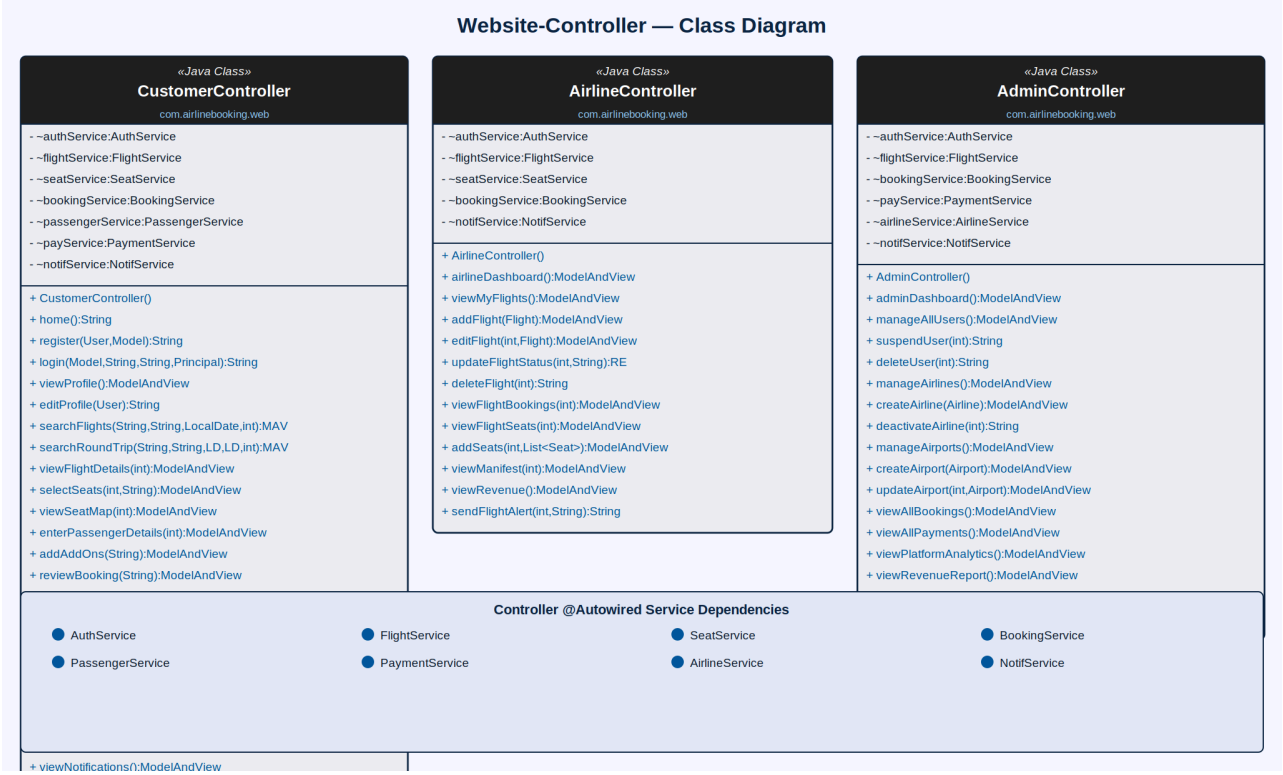

Figure 10: Website-Controller — Class Diagram

Controller	Type	Key Methods
CustomerController	Java Class (MVC)	home(), register(), login(), viewProfile(), editProfile(), searchFlights(), searchRoundTrip(), viewFlightDetails(), selectSeats(), viewSeatMap(), enterPassengerDetails(), addAddOns(), reviewBooking(), checkout(), makePayment(), bookingConfirmation(), viewMyBookings(), viewBookingDetail(), cancelBooking(), downloadETicket(), webCheckin(), viewNotifications(), logout()
AirlineController	Java Class (MVC)	airlineDashboard(), viewMyFlights(), addFlight(), editFlight(), updateFlightStatus(), deleteFlight(), viewFlightBookings(), viewFlightSeats(),

Controller	Type	Key Methods
		addSeats(), viewManifest(), viewRevenue(), sendFlightAlert()
AdminController	Java Class (MVC)	adminDashboard(), manageAllUsers(), suspendUser(), deleteUser(), manageAirlines(), createAirline(), deactivateAirline(), manageAirports(), createAirport(), updateAirport(), viewAllBookings(), viewAllPayments(), viewPlatformAnalytics(), viewRevenueReport(), sendPlatformNotification(), viewAuditLogs()

5. Microservices Architecture Overview

SkyBooker is structured as a set of independently deployable microservices following the EShoppingZone reference pattern. Each service owns its own database schema, REST API contract, service interface, and business logic. Synchronous inter-service calls use RestTemplate; asynchronous notification dispatch and flight status sync are handled via RabbitMQ and Spring Scheduler.

Microservice	Base Package	Primary Domain
auth-service	com.skybooker.auth	User accounts, JWT, OAuth2 (Google), passport/nationality details
flight-service	com.skybooker.flight	Flight schedules, search (one-way/round-trip), status, seat counters
seat-service	com.skybooker.seat	Seat map, hold/release/confirm lifecycle, class-based pricing
booking-service	com.skybooker.booking	Booking lifecycle, PNR generation, fare calculation, ancillary add-ons
passenger-service	com.skybooker.passenger	Passenger details, ticket numbers, seat assignment, check-in status
payment-service	com.skybooker.payment	Payment initiation, gateway callback, refunds, revenue aggregation
notification-service	com.skybooker.notification	In-app, email, SMS alerts; booking confirmation; flight status alerts
airline-service	com.skybooker.airline	Airline and airport master data, IATA codes, airport search autocomplete
skybooker-web	com.skybooker.web	Spring MVC controllers, Thymeleaf view rendering

6. Non-Functional Requirements

Category	Requirement
Performance	Flight search results must load within 2 seconds for 50,000 concurrent users; Redis caches popular route search results with 5-minute TTL
Scalability	Each microservice independently scalable via Docker/Kubernetes; Flight-Service and Booking-Service autoscale during peak booking windows
Security	Passwords stored as bcrypt hashes; JWT tokens expire in 24 hours; OAuth2 for Google login; HTTPS enforced; PCI-DSS compliance via tokenised payment gateway
Availability	99.9% uptime SLA; health-check endpoints per service; graceful degradation if notification service is temporarily unavailable
Data Integrity	Seat hold and booking creation are atomic using database-level optimistic locking to prevent double-booking of the same seat
Seat Hold Expiry	Seat holds expire after 15 minutes if payment is not completed; a Spring @Scheduled job runs every 2 minutes to release expired holds
PNR Uniqueness	PNR codes are 6-character alphanumeric strings generated using a collision-checked unique constraint in the database
Fare Accuracy	Dynamic pricing: base fare varies by demand; price is locked at the time of seat hold and does not change during the 15-minute payment window
Audit Trail	All booking status transitions, payment events, and admin actions are logged with actor, timestamp, and before/after values
Usability	Responsive UI on desktop, tablet, and mobile; seat map rendered as an SVG grid; WCAG 2.1 AA accessibility compliance

7. Proposed Technology Stack

Layer	Technology
Backend Framework	Java Spring Boot (Spring MVC, Spring Security, Spring Data JPA)
Authentication	JWT (JSON Web Tokens), Spring Security OAuth2, Google OAuth2
Database	MySQL (primary relational store per service); Redis (seat hold cache, flight search cache, session store)
Payment Gateway	Razorpay or Stripe (webhook-based callback for payment confirmation); PCI-DSS compliant tokenisation
PDF Generation	iText 7 or JasperReports for e-ticket and boarding pass PDF generation with QR code embed
QR Code	ZXing (Zebra Crossing) Java library for generating barcode/QR on boarding passes
Seat Map UI	SVG-based interactive seat map rendered server-side (Thymeleaf) or client-side (React with D3.js)
File Storage	AWS S3 for e-ticket PDF storage; CloudFront CDN for fast delivery; pre-signed URL expiry 24h
Notifications	JavaMailSender via AWS SES for email; Twilio or AWS SNS for SMS; Firebase FCM for mobile push
Messaging	RabbitMQ for async notification fan-out; flight status update events; booking confirmation email queue
Scheduled Jobs	Spring @Scheduled + Quartz: seat hold release (2-min), check-in reminder (hourly), no-show detection (post-departure)
Frontend	Thymeleaf with Bootstrap 5 and JavaScript; or React.js SPA with Tailwind CSS for the booking flow
Containerisation	Docker with Docker Compose; Kubernetes for production autoscaling
API Documentation	Swagger / OpenAPI 3.0 for all REST endpoints
CI/CD	GitHub Actions pipeline: test → Docker build → ECR push → Kubernetes rolling deploy

8. Glossary

Term	Definition
PNR	Passenger Name Record — a unique 6-character alphanumeric booking reference number assigned to every confirmed reservation
IATA Code	International Air Transport Association code — a 2-3 character identifier for airlines (e.g., 6E for IndiGo) and 3-character code for airports (e.g., DEL for Delhi)
ICAO Code	International Civil Aviation Organization code — a 4-character identifier for airlines and airports used in air traffic control
Fare Class	The category of ticket (Economy, Business, First) that determines pricing, seat features, baggage allowance, and refund policy
Seat Hold	A temporary 15-minute reservation of a seat during the payment process, preventing other users from booking the same seat
Ancillary Service	An optional paid add-on to a flight booking such as extra baggage allowance, seat upgrade, or meal preference
Web Check-In	The online process of confirming attendance on a flight, available 24 hours to 1 hour before scheduled departure time
Boarding Pass	A travel document (paper or electronic) issued after check-in confirming a passenger's right to board a specific flight
E-Ticket	An electronic flight ticket stored in the airline's booking system, replacing a physical paper ticket — delivered as PDF via email
Manifest	The complete list of passengers booked on a specific flight, including seat assignments and check-in status
Seat Map	An interactive visual representation of the aircraft's seating layout showing available, held, and occupied seats by class
PriceMultiplier	A decimal factor applied to the base fare for specific seat types (e.g., 1.2 for extra legroom, 1.5 for bulkhead seats)
FareSummary	A data transfer object containing an itemised breakdown of: base fare, taxes, fuel surcharge, and ancillary add-on costs
Optimistic Locking	A concurrency strategy where seat hold and booking creation include a version check to prevent two users from booking the same seat simultaneously
No-Show	A booking status assigned when a passenger fails to board the aircraft and has not cancelled prior to departure
Refund	A monetary reimbursement to the original payment mode for a cancelled booking, subject to the airline's cancellation policy and charge schedule